

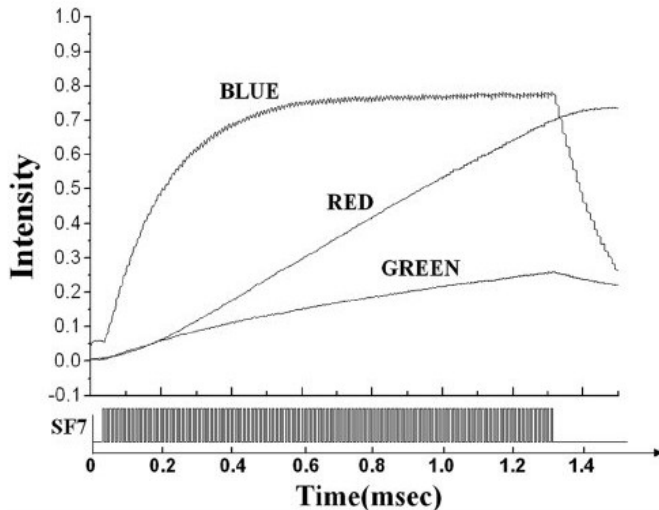
Laboratory 0xA

Expected delivery of **lab_10.zip** must include:

- Solutions of exercises 1, and 2.
- This file, filled with information and possibly compiled in a pdf format.

Delivery deadline: 20/12/2025

Exercise 1) Implement a system on the LANDTIGER board that can tune the brightness of an LED by making use of TIMERS.

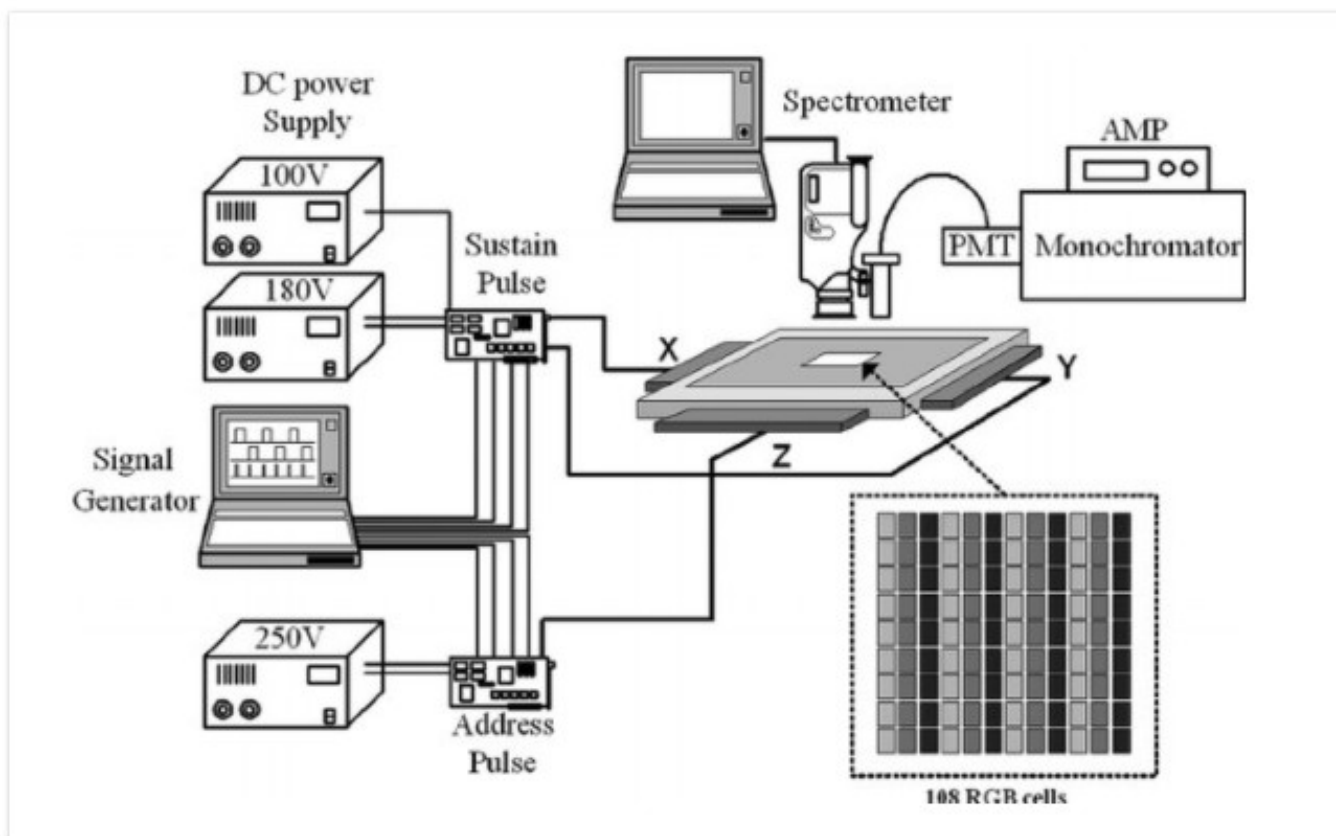


Dimming a LED is done by appropriately turning it ON and OFF. Usually, the maximum brightness (100% brightness) is achieved when it is ON for at least a period T period. Such T time value can be often found in the LED datasheet (when available). Please see an example, where also blue and green LEDs are considered, which shows a different behaviour (non-linear).

For LANDTIGER red LEDs, the time to saturate should be in the order of few milliseconds;

during this time the brightness increases almost linearly from no light to the maximum luminescence.

To determine the brightness of a LED is quite difficult and require expensive machineries (as shown below). We suggest you perform just a visual comparison with a setup where a LED modulated while another is always on.



In this lab, you are required to experiment with LEDs dimming using Pulse Width Modulation (PWM) via software, i.e., achieved by using GPIO outputs (on LEDs) and timers.

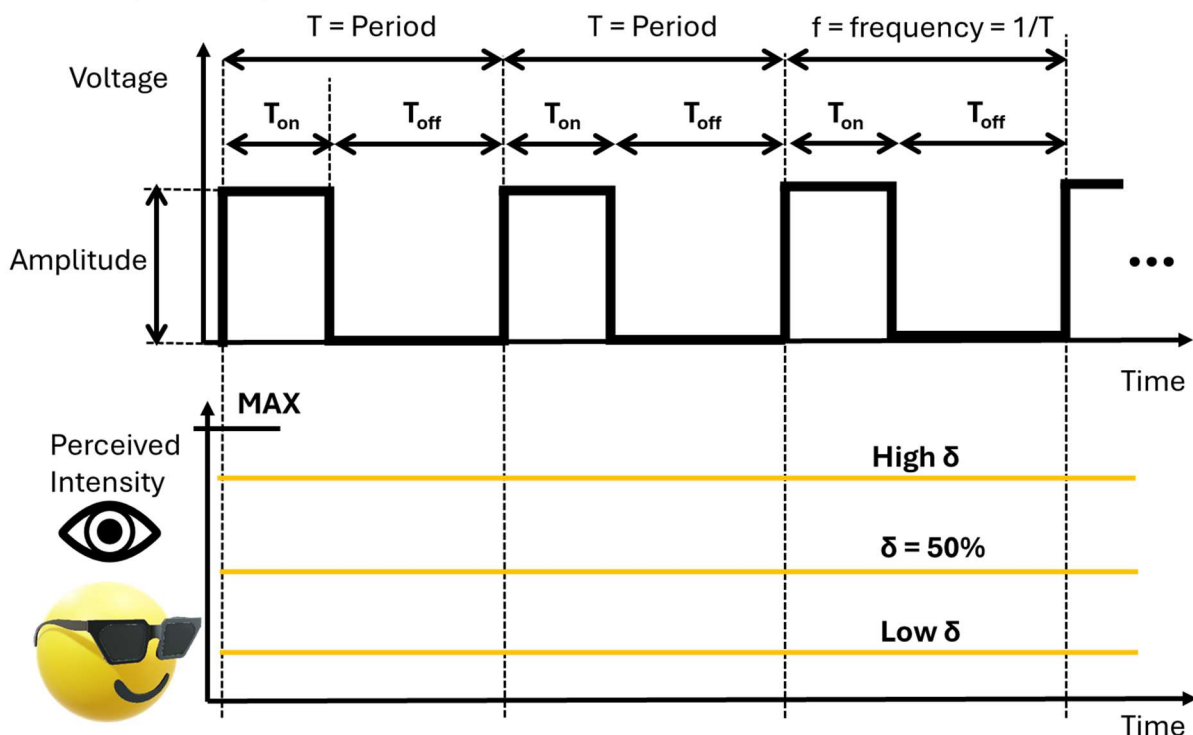
There exist specific peripherals for generating PWM signals (even in the LPC1768 SoC, look for them!), but they are out of the scope for this lab.

Unfortunately, a spectrometer for measuring LEDs intensity is not available in the lab.

We are going to use a rudimental non-linear photometric sensing device, **the human eyes** 🧐.

The maximum frequency at which a human eye can perceive a light source (like an LED) flickering is related to the concept of the flicker fusion threshold. This is the frequency above which the human eye perceives a flickering light as a steady, continuous glow. For most people, the flicker fusion threshold is around 60 Hz to 90 Hz under typical lighting conditions. Thus, even if human eyes are the souls' mirror, they do not work good as intensity sensing device for digital signals!

The following figure represents how a PWM signal (based on a square wave) is build and how it is perceived by human eyes.



Remember that:

- Period (T) or frequency ($1/T$) is the amount of time for which the digital signal repeats itself.
- Time On (T_{on}) is the amount of time for which the digital signal stays at the logic value **ONE** (from an electrical perspective, it is the maximum voltage amplitude your GPIOs pins can reach).
- Time Off (T_{off}) is the amount of time for which the digital signal stays at the logic value **ZERO**.
- The duty cycle is defined as the amount of time the digital signal stays at the logic value ONE divided by the Total amount of time the digital signal repeats itself.

In other words:

$$\delta = \frac{T_{on}}{T} [\%] \text{ where } T = T_{on} + T_{off}$$

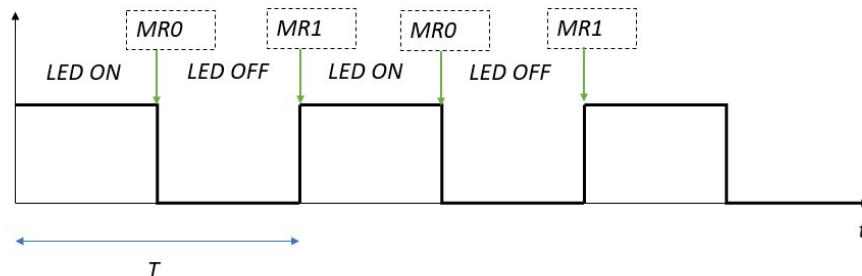
A higher duty cycle results into a higher perceived intensity of the LED. A lower duty cycle results into a lower perceived intensity.

Therefore, to achieve a 50% brightness (perceived intensity), for instance, the LED would have to be ON half of the time in each T period ($\delta = 50\%$).

Note: When a LED is blinking with period T without mention the duty cycle, it implies that the duty cycle is 50 %.

To solve the exercise:

1. Select one of the available LEDs, and use two Match Registers (MR0 and MR1) of TIMER 2 to synchronize the two phases. Specifically, it is necessary to set the two Match Registers so that, when TIMER 2 reaches MR0 the LED turns OFF; when the timer reaches the value held in MR1, the LED must light up.
 - a. Selects a frequency that **avoid** LED flickering.



2. Select another of the available LEDs, and use two Match Registers (MR0 and MR1) of TIMER 3 to synchronize the two phases (as before).
 - a. Selects a frequency that **does not avoid** LED flickering.
3. Compare the TWO LEDS with the same duty cycle (50%) and different frequency.

What is happening?

At lower frequency the LED visibly alternates between on and off states, producing a clear flickering effect because the eye can detect the switching. At higher frequency the LED is still turning on and off with the same duty cycle, but the transitions occur so quickly that the eye cannot follow them, and instead it perceives a steady light of half the maximum brightness. In other words, the duty cycle determines the average intensity, while the frequency determines whether the LED appears to flicker or to shine continuously.

Fill in the table the required brightness for Timer 2.

Brightness [%]	MR0	MR1	Time ON	Time OFF	Period	Frequency
25	0x196E6	0x65B9A	0,004 s	0,012 s	0,016 s	60 Hz
50	0x32DCD	0x65B9A	0,008 s	0,008 s	0,016 s	60 Hz
75	0x4C4B4	0x65B9A	0,012 s	0,004 s	0,016 s	60 Hz
100	0x65B9A	0x65B9A	0,016 s	0 s	0,016 s	60 Hz

Fill in the table the required brightness for Timer 3.

Brightness [%]	MR0	MR1	Time ON	Time OFF	Period	Frequency
25	0x2625A	0x98968	0,006 s	0,019 s	0,025 s	40 Hz
50	0x4C4B4	0x98968	0,012 s	0,012 s	0,025 s	40 Hz
75	0x7270E	0x98968	0,019 s	0,006 s	0,025 s	40 Hz
100	0x98968	0x98968	0,025 s	0 s	0,025 s	40 Hz

NOTE:

1. To use TIMER2 and TIMER3 functionalities, you must extend the available libraries and create the appropriate functions.

Exercise 2) Starting from the project of Exercise 1, enhance the firmware with the following features:

- Upon reset, two counters (TIMER1 and TIMER2) are activated, both programmed cyclically, counting 211 milliseconds and 307 milliseconds respectively.
- The program must first count the number of times a “collision” occurs between the interrupt requests of TIMER1 and TIMER2 (in that order of arrival). The number of collisions must be saved in a variable called NUMBER, which is an 8-bit unsigned integer. When the value 255 (the maximum representable on 8 bits) is reached, the value of NUMBER must be reset to 0.
- The LEDs show the current value of NUMBER.
- Pressing KEY1 the value of NUMBER shall be decreased by 1.

The highest priority should be given to the collision between timers.

Complete the table below:

Interrupt	Priority value
Timer 1	0
Timer 2	1
KEY1	2

What is happening to LED counting if you repeatedly press KEY1?

In practice, pressing KEY1 does not change the LED count. This happens because KEY1 has the lowest interrupt priority in the NVIC configuration. Since TIMER1 and TIMER2 generate periodic interrupts at short intervals, they continuously pre-empt the CPU, preventing the KEY1 interrupt service routine from being executed. As a result, the decrement function associated with KEY1 is never observed on the board.

How can you solve the issue?

The issue can be solved by adjusting the NVIC interrupt priorities. By assigning a higher priority to KEY1 compared to the timers, the CPU will service the button interrupt even when TIMER1 and TIMER2 are active. Alternatively, reducing the timer interrupt frequency or delegating button handling to the Repetitive Interrupt Timer can also ensure that KEY1 is correctly executed.